

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

20/06/2025

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation: _____
☐ No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 18 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

An international airport uses an automated baggage handling system (BHS) to route passenger luggage from check-in counters to aircraft and from arriving flights to the baggage claim area. The system relies on programmable logic controllers (PLCs) to control conveyor belts, diverters, and scanners. These components are coordinated and monitored by a central SCADA-like system, operated by airport staff in a control room.

The BHS communicates with:

- Airline databases to match baggage to flights.
- Airport IT systems are used to obtain passenger and gate information.
- Barcode and RFID scanners to track luggage movement.
- Remote maintenance systems for software updates and troubleshooting.

All components are connected via the airport's internal network, which also provides restricted remote access for maintenance personnel. In some cases, temporary Wi-Fi access is enabled for system diagnostics. Operational continuity is critical, especially during peak traffic.

Answer the following questions, considering the security implications in this specific scenario only:

[1.5 points] 1. Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats, threat agents. All elements must be motivated.

Asset	Threat	Threat Agent
Baggage Routing Logic / Control System	An attacker could manipulate PLC commands or SCADA logic to misroute baggage intentionally (e.g., sending bags to the wrong flights or stopping the conveyor system entirely). Impact: Lost or delayed baggage, flight delays, and reputational damage to the airport and airlines.	Cybercriminals seeking ransom in exchange for system restoration. Insiders (e.g., disgruntled employees or contractors) with authorized access to internal systems. Motivation: High operational impact and potential for extortion or disruption.
Passenger and Flight Assignment Data	Access to airline databases and passenger-luggage associations could be used to: • Target specific individuals (VIPs, officials). • Extract personal data for identity theft. • Launch misinformation or social engineering attacks (e.g., rerouting a bag under false identity).	Hactivists, interested in targeting specific flights or airlines. Nation-state actors for espionage or surveillance purposes. Motivation: Sensitive data, high-profile targets, or political interests.

[1 point] 2. Suggest at least two potential attack surfaces for the infrastructure under analysis.

Attack surface	Motivation
----------------	------------

Internal Network & Remote Access Interfaces	Internal network interconnects critical systems (PLCs, SCADA, airline databases). If attackers breach the network or exploit poorly secured remote access interfaces (e.g., VPN, RDP), they could issue commands or extract data.
PLCs and SCADA System	PLCs may run old firmware or use insecure protocols (e.g., Modbus, S7) and lack strong authentication. Exploiting these could allow direct control of the mechanical system.

[1.5 points] During a system audit, it is discovered that the SCADA system authenticates PLCs using self-signed certificates stored locally on each device. The SCADA controller accepts any certificate as long as it appears "well-formed" and contains a known device ID.

- Identify and explain two major security weaknesses in this authentication approach.
- Propose at least one modification or alternative that would improve the system's trust model.

Weaknesses	Suggested Improvement
No Trust Anchoring or CA Verification Self-signed certificates provide no chain of trust. Any attacker with access to the network can generate a valid-looking certificate with a forged device ID. The SCADA system does not validate whether the certificate comes from a legitimate source (i.e., no CA or trust store is used).	Implement Certificate Authority (CA)-Based PKI
Storing certificates locally and accepting them permanently means there is no mechanism for revoking compromised certificates. If a device is retired or cloned, its credentials may remain accepted indefinitely.	Introduce Device Attestation or TPM-Based Identity

Exercise 2 - Introduction to Cryptography (6 points)

[4 points] Consider a messaging system which wants to mimic the current paper-based mail system, and is willing to deliver, via multiple hops, asynchronous messages from one host to another, through a public network (If you are familiar, simply consider the classic e-mail application level protocols SMTP+POP3).

The equivalents of the post offices, i.e., the servers acting as message relays want to ensure the integrity of the messages they are delivering, therefore, proceed in the following fashion.

- Upon relaying a message, they append a timestamp, to ensure freshness, and compute the digest of a cryptographically sound hash on the message and the timestamp. They append the message+timestamp digest to the time stamped message itself and relay it
- Upon receiving a message to be relayed the server computes the hash digest on the time stamped message, and checks that the digest matches with the one appended. If it does not, it discards the message, otherwise it proceeds to check that the timestamp is actually in the past.

Assume that the client-to-final destination relay and the sender-to initial relay communications are not affected by any vulnerability.

Argue why the proposed e-mail system provides the desired message integrity and replay attack resistance, providing a proof sketch, or provide an attack against integrity and propose a cryptographic protocol-level fix.

In the analysis, you may assume that the relays mutually trust each other fully, and want to minimize their computational load for the checks (hence the hashes).

The described system does not provide integrity. Indeed, an attacker may simply take any message in transit, and alter it, re-timestamp it to its own will, and compute the resulting digest. Injecting this fresh message in the traffic will have the relays recompute a matching digest. A simple and effective fix is represented by computing a MAC over the relayed messages and timestamps, with a key shared by the relays. The MAC tag is computed before relaying and checked on reception.

[2 points] I have the habit of picking the passwords according to my diceware criterion, rolling ten (fair) twenty-sided dice and picking the resulting concatenation of the digits on the dice as a password. I therefore obtain a password which is between 10 and 20 randomly selected digits. Quantify the hardness of cracking the password as the number of attempts to be made, on average. Is this a secure choice to ward-off cracking?

The number of attempts is, in the worst case 20^{10} , and, in the average case $2^9 \cdot 10^{10}$ (about 2^{43}), which is definitely low to ward off cracking attacks.

Exercise 3 - Binary Vulnerabilities (6 points)

Isabella, a provola lover and chief cheese artisan at Provola by Teramo, has developed a system to manage provola orders. However, the system seems to have some issues...

```
1. #include <stdio.h>
2. #include <string.h>
3.
4. #define PROVOLE_COUNT 5
5.
6. typedef struct {
7.     char name[30];
8.     float price;
9. } ProvolaDB;
10.
11. ProvolaDB provola_database[PROVOLE_COUNT] = {
12.     {"PAf", 8.50}, // Smoked Provola
13.     {"PPi", 9.50}, // Spicy Provola
14.     {"PDo", 7.20}, // Mild Provola
15.     {"PBU", 10.00}, // Buffalo Milk Provola
16.     {"PTa", 12.50} // Truffle Provola
17. };
18.
19. typedef struct {
20.     char provola_type[4];
21.     char cashier_name[32];
22.     unsigned int size_provola;
23.     unsigned int size_cashier;
24. } Session;
25.
26. int main() {
27.     Session session = {0};
28.
29.     session.size_provola = 4;
30.     session.size_cashier = 32;
31.
32.     // Ask for cashier name
33.     printf("Insert the name of the cashier: ");
34.     for (int i = 0; i <= session.size_cashier; i++) {
35.         char ch = getchar();
36.         if (ch == '\n') {
37.             break;
38.         }
39.         session.cashier_name[i] = ch;
40.     }
41.
42.     printf("Ciao %s! 🍕\n", session.cashier_name);
43.
44.     // Ask for provola name
45.     printf("Insert the provola type: ");
46.     fgets(session.provola_type, session.size_provola, stdin);
47.
48.     // Search for provola
49.     int found = 0;
50.     for (int i = 0; i < PROVOLE_COUNT; i++) {
51.         if (strcmp(session.provola_type, provola_database[i].name) == 0) {
52.             printf("Price of '%s': €%.2f\n", provola_database[i].name,
provola_database[i].price);
53.             found = 1;
54.             break;
```

```

55.     }
56.     }
57.
58.     if (!found) {
59.         printf(session.provola_type);
60.         puts(" is not a valid provola type. Please try again.");
61.     }
62.
63.     return 0;
64. }

```

The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).

Man

getchar(): reads the next character from stream and returns it [...]

*fgets(char *s, int size, FILE *stream): reads in at most one less than size characters from stream and stores them into the buffer pointed to by s. Reading stops after an EOF or a newline. If a newline is read, it is stored into the buffer. A terminating null byte ('\0') is stored after the last character in the buffer.*

[1 point] 1. Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code and specify a valid solution to solve the issue.

Vulnerability	Line	If the vulnerability is present, motivate and modify the line to solve the detected vulnerability
Buffer Overflow	Lines 34-40 [0.25]	See Slides [0.25] Note: The overflow at lines 34–40 enables another overflow at line 46. While explicitly specifying this wasn't strictly necessary, it's appreciated.
Format String	[0.25] Line 59	See Slides [0.25]

[3 points] 2. Santeramo, the main competitor of Provola by Teramo, managed to execute the following 8-byte **shellcode**, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

You have been hired by Isabella as a security analyst for Provola by Teramo. You are required to evaluate which vulnerability might have been exploited.

- If a BOF can be used to perform the attack, describe it in detail. If not, explain why it cannot.
- If a format string vulnerability can be used to perform the attack, describe it in detail. If not, explain why it cannot.
- If both can be used, describe both exploits in detail.
- If neither can be used, explain why.

A one-byte overflow in the for loop at lines 34–40 can be used to overwrite the size_provola variable. This allows us to:

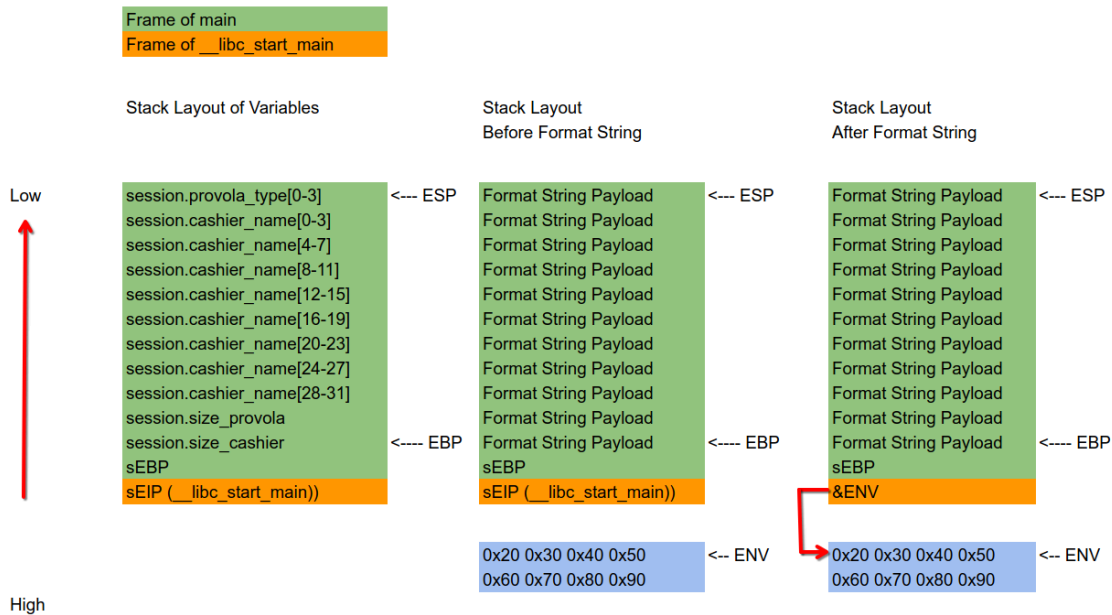
- Unlock a second overflow at line 46. We can exploit this to overwrite the saved EIP (sEIP) with the address of the injected shellcode. The shellcode can be placed at a higher or lower memory address relative to sEIP, in environment variables, and so on.*
- Make the format string vulnerability exploitable. Previously limited to only 4 bytes, the overflow removes this restriction. We can now use the format string to overwrite the sEIP (the target - where to write) with the address of the shellcode (what to write). The simplest approach is to store the shellcode in environment variables. Alternatively, with sufficient space optimization, the shellcode can be placed in the memory region between the format string payload (which overflows through the Session struct) and the sEIP.*

[1 point] 3. Show the effect of one of the identified vulnerabilities on memory. If you have found at least one exploit that enables execution of the shellcode described in point 2, choose **one of them** and illustrate the **stack layout both before and after** the execution of the most relevant line for that exploit, including:

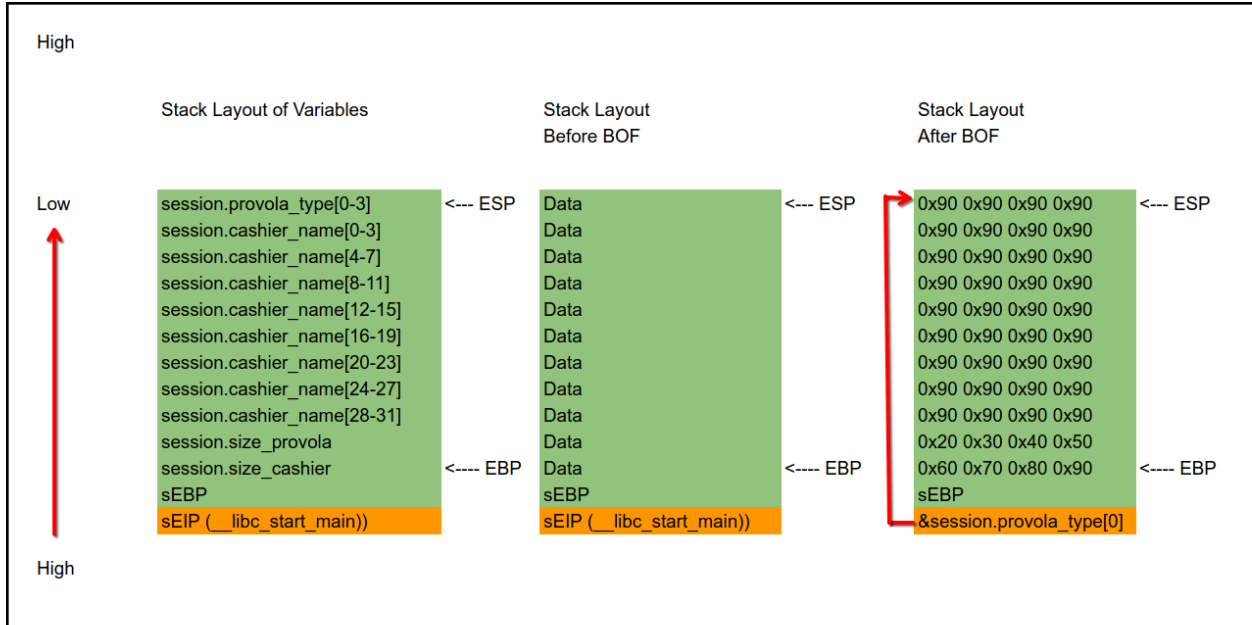
- The **direction of memory growth** (the stack grows downward) and the **relative address order** (from high to low);
- The **name of each relevant allocated variable** in the stack frame;
- The **contents of relevant registers**, such as **EBP** and **ESP**;
- The **stack frames of the current function** and any **callers**;
- The **contents of the caller's frame**, including saved return addresses and local variables.

*NOTICE: We did not consider variables such as **found**, **i**, and **c**. Since the placement of these variables is compiler-dependent, we will accept both solutions that include or exclude them in the stack layout, provided the exploit rationale is correct.*

Solution for Format String:



Solution for BOF



[1 points] 4. Consider ONLY the correctly implemented “Stack Canary” mitigation technique (i.e, randomly generated and non-guessable). How does your answer to point 2 change? Motivate your answer.

You can no longer use the BOF exploit, as the code does not provide any way to leak the stack canary before overwriting the sEIP variable.

However, you can still use the format string exploit to directly modify sEIP.

Exercise 4 - Web Vulnerabilities (6 points)

iTransfer is a new web service that makes sharing files simple. Upload the files you want to send, generate a download link, and share it with your friends.

The relevant code of the application is the following:

```
00 @app.route('/upload', methods=['POST'])
01 def upload(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("login"))
04     user_id = session["user_id"]
05     file = request.files.get("file")
06     if not file or file.filename == "":
07         return render_template("home.html", error="No file uploaded!")
08     uuid = get_random_uuid()
09     q = "INSERT INTO files VALUES ('" + uuid + "', " + user_id + ", '" +
        file.filename + "');"
10     db_execute_query(q)
11     with open("uploads/" + str(user_id) + "/" + uuid, "wb") as f:
12         f.write(file.content) # Just writing the content into a file
13     return render_template("files.html", success="New file UUID: " + uuid)
14
15 @app.route('/download', methods=['GET'])
16 def download(request, session):
17     if not is_session_valid(session):
18         return redirect(url_for("login"))
19     if "owner_id" not in request.args.keys():
20         return render_template("home.html", error="No owner ID specified!")
21     owner_id = request.args["owner_id"]
22     if "uuid" not in request.args.keys():
23         return render_template("home.html", error="No UUID specified!")
24     uuid = request.args["uuid"]
25     q = "SELECT * FROM files WHERE uuid = '" + uuid + "' AND owner_id = '" +
        owner_id + "';"
26     result = db_execute_query(q)
27     if len(result) != 1:
28         return render_template("home.html", error="Invalid UUID!")
29     filename = result[0]["filename"]
30     uuid = result[0]["uuid"]
31     filepath = "uploads/" + owner_id + "/" + uuid
32     return render_template("download.html", filename=filename, filepath=filepath)
...
50 <html> <!-- download.html -->
...
60     <h1>{{ filename }}</h1>
61     <a href="/files?filepath={{filepath}}">Download!</a>
...
70 </html>
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)
--	----------------------	----------------------	-------------------

Table: **files**

owner_id (Primary Key) (int) (Reference: users.user_id)	uuid (Primary Key) (string)	filename (string)
---	--------------------------------	----------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query** executes queries without prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ...; INSERT INTO ...;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.

More details:

- The function **render_template** generates an HTML page from an .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL for the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided URL.
- The function **db_execute_query** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.
- The function **get_random_uuid** returns a random UUID (e.g., `dd9bfc99-5c37-456a-b943-9a2e59b60324`)
- The function **open** returns a file descriptor that can be used to read/write from/to the file. The function **open** has 2 parameters: the path where the file is or should be written on the filesystem, and the mode (read or write). For instance, `open("/tmp/test", "wb")` is going to create a file called test under /tmp and open it for binary writing.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
SQL Injection (SQL-I)	<i>Line 9: An attacker can insert data into the database. The attacker can also read data from the db by opening a subquery Line 25: An attacker can read data from the db</i>	<i>To fix the vulnerability just employ prepared statements</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>The name of the file can be used for an XSS. It is a stored XSS.</i>	<i>Escaping/whitelisting (see slides)</i>
Cross-site request forgery (CSRF)	<i>The function upload does not have a CSRF, and it is a state-changing action. Therefore, there's a CSRF. An attacker can potentially upload files for another user.</i>	<i>To fix the vulnerability just employ CSRF tokens</i>

[1.5 points] 2. One of your friends sent you a link to iTransfer.com. You open it, and a pop-up says, "I told you not to trust me". Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The attacker uploaded a file with the following name:

<SCRIPT>alert("I told you not to trust me")</SCRIPT>

And then the attacker just provided the link to the file to the victim

[1.5 points] 3. For some reason, your password has been stolen, and all your files are gone. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The attacker used one of the two injections to leak the password. The easiest one is probably the second, and the exploit can be the following:

```
uuid = " ' UNION ALL SELECT 1, password, password FROM users WHERE username =  
'<my_username>'; -- "
```

The password will appear both in the download link and in the filename

Exercise 5 - Network Security (6 points)

As every year the Best Salesman Award is to be assigned: a proper competition is put in place in which every seller who wants to participate needs to upload each contract he/she is capable of closing so that, in the end, a proper ranking can be generated based on the sales that were made.

The two best candidates are **Paolo Bitta (IP address: 78.216.196.6)** and his arch-rival **Michele Carminati (IP address: 237.2.33.165)** from Digitex, who are winning thanks to the great number of contracts they were able to close, vending C-14s products. Now it is almost the end of the competition, but Paolo Bitta cannot stand losing, so he is preparing quite a scheme to ensure his win. He wants to inject a malicious software called **DammiSoloUnMinuto.exe** as if he were Carminati, so that, once the malware is uploaded on the server, it will overwrite all Carminati's contracts with random songs by Pooh, ruining Carminati score.

The competition is based on a **remote server (IP address: 136.233.6.188)** which provides a custom service: each user (i.e., each salesman who wants to participate) is associated with a unique IP address and will connect to the server via a custom implemented TCP/IP protocol, where the ISN for the communication is defined as the sum of the digits of the current timestamp.

[2 point] 1. What type of attack can Paolo Bitta implement to put in place his scheme? Motivate your answer and describe how the attack works.

Blind tcp hijacking, by spoofing Carminati's IP and opening a connection with the server. He can predict ISN numbers and then simply upload the malware. See slides for more details.

While Paolo and Luca are planning this malicious plan, Silvano overhears them and discovers the vulnerability they want to exploit: after consulting his mom, he decides to contact **Best Seller Award inc.** and securely disclose the vulnerability. So now the service is up and running nicely, with the default TCP/IP protocol (i.e., not custom-implemented one). Yet, on the last day of the competition, Paolo is still in position one, so everything seems to be leading to his victory.

[2 points] 2. Can Paolo still do something to avoid risking losing his position? How can he implement it?

DoS attack against the server with a lot of half-opened connections so that no one else can upload his/her contracts anymore, freezing the rankings.

[1 point] 3. Can this last attack be avoided server-side (without adding extra network infrastructure)?

SYN-cookies allow to solve such problems. See slides for more details.

[1 point] 4. By considering a stateful packet filter firewall implemented in the same server network, **can** previous attacks be mitigated? **Motivate your answer.**

With a stateful packet filter firewall several things can be done.

The first attack can be mitigated with NAT, since it is possible to change and manipulate ISN numbers, making the custom-implemented TCP/IP protocol secure.

Also it is possible to drop incoming packets from some specified IPs but this might also limit legitimate traffic coming from them. Besides, if the attacker uses a botnet for a DDoS, then even with a firewall nothing could be done.

The DoS attack cannot be mitigated via firewall.

Exercise 6 - Malware (6 points)

During a recent audit of prototypes developed by **Neurobyte Industries**, security engineers uncovered the presence of a suspicious memory-resident routine embedded within a performance testing binary.

This code appears to dynamically construct and execute a patched version of its own embedded bytecode:

```
uint8_t evil_bytecode[] = {
    0x48, 0xC7, 0xC0, 0x05, 0x00, 0x00, 0x00,
    0x48, 0x83, 0xC0, 0x03,
    0xC3
};

int main() {
    sleep(500);

    size_t original_len = sizeof(evil_bytecode);

    csh handle;
    cs_insn *insn;
    cs_open(CS_ARCH_X86, CS_MODE_64, &handle);

    size_t count = cs_disasm(handle, evil_bytecode, original_len, (uint64_t)evil_bytecode, 0, &insn);

    // Allocate RWX memory
    size_t patch_len = original_len * 2;
    void *exec = mmap(NULL, patch_len, PROT_READ | PROT_WRITE | PROT_EXEC,
        MAP_ANONYMOUS | MAP_PRIVATE, -1, 0);

    // Write original instructions + NOP after each
```

```
char *out = exec;
for (size_t i = 0; i < count; i++) {
    memcpy(out, evil_bytecode + (insn[i].address - (uint64_t)evil_bytecode), insn[i].size);
    out += insn[i].size;
    *out++ = 0x90;
    if (insn[i].id == X86_INS_RET)
        break;
}

// Executing evil_bytecode...
int (*fn)() = (int (*)(()))exec;
fn();
}
```

1. [2 Points] Identify and explain any anti-analysis techniques used in the code. For each technique, explain how it works, what type of analysis (e.g., static or dynamic) it attempts to evade.

[1 point] **Technique 1** – Time-based evasion using `sleep(500)` introduces a delay in execution to bypass sandbox timeouts. This technique aims to evade dynamic analysis and sandbox-based detection by delaying observable malicious behavior.

[1 point] **Technique 2** – This technique inserts NOP instructions at runtime, placing them after each legitimate instruction to alter the in-memory layout of the code. However, unlike polymorphic or metamorphic techniques, it does not produce a different binary for each instance. Specifically, the payload is not encrypted with a unique key per sample (as in polymorphism), nor does it include semantic changes or instruction mutation (as in metamorphism). Since the static binary remains unchanged, it is ineffective against static signature detection. Moreover, the addition of NOPs does not conceal the malware's behavior during execution, making it equally ineffective against dynamic analysis.

2. [2 Points] It's clear that at least one of the previously indicated techniques is not correctly implemented. For each one, explain: (i) why it is not correct; (ii) how an attacker could improve it.

[1 point] As explained in the previous answer, the second technique is not effective because it fails to evade both static and dynamic analysis. From a detection perspective, it can be easily identified by static signature detection, as the binary remains unchanged across all samples—the modification occurs only at runtime. Additionally, it is ineffective against behavioral detection, since at runtime the code merely inserts a series of NOP instructions, which do not conceal any aspect of the malware's behavior (they are, after all, just NOPs).

[1 point] **Improvements:**

- To make it **polymorphic**, encrypt the `evil_bytecode` with a different key per sample and

decrypt it at runtime.

- To make it **metamorphic**, generate syntactically different but semantically equivalent versions of the payload (e.g., instruction reordering, register substitution, equivalent operations like `xor rax, rax; add rax, 5` instead of `mov rax, 5`) or randomly put NOP at each propagation.

[0.5 extra point] Assuming a sleep duration in ms, also the timebomb was not so effective since too brief with respect to a standard sandbox duration analysis.

3. [2 Points] Describe how you would **analyze this malware sample**. Your answer should include:

- Type(s) of analysis you would use (static/dynamic/etc.).
- Key elements you'd observe or extract to understand the malware's behavior.

[2 points] As discussed in the previous answers, this malware does not implement any anti-analysis techniques. Therefore, it can be effectively examined through static analysis— by decompiling the entire sample or focusing on the `evil_payload`. Alternatively, dynamic analysis is also applicable: the sample can be executed in a sandbox environment to observe its interactions with the system (e.g., system calls, network connections, file operations, etc.).

Notes:

- Simply writing "static and dynamic analysis" without providing appropriate justification and description results in 0 points. The question asked for analysis methods, not detection techniques.
- Confusing static analysis with signature-based detection, or discussing signature extraction from dynamic analysis, was considered a misunderstanding and penalized with -1 point.